

CONTRATO ZNVE v1.3 — Sistema de Gestión Escolar

Cambios respecto a v1.2

Cambio	Detalle
Base de datos	PostgreSQL 18 aprobado; la persistencia pasa de agnóstica a específica.
Imágenes	Nuevo módulo de imágenes: archivos en carpeta local y solo metadatos en la base de datos.
Autenticación	Nuevo contrato explícito (<code>User</code> , login, logout). Se elimina <code>Role.GUARDIAN</code> : los tutores no tienen cuenta y consultan su matrícula con <code>enrollmentId</code> y email.
Eventos de dominio	Se persisten en una tabla outbox dentro de PostgreSQL, sin broker externo.
<code>STUDENT_PHOTO</code>	Se reincorpora la fotografía del alumno, que v1.1 había descartado. Queda pendiente confirmarla.

0. ALCANCE Y STACK

Capa	Decisión
Plataforma	Aplicación web
Backend	Node.js 24 LTS, TypeScript ejecutado nativamente (<i>type stripping</i>), <code>node:http</code>
Base de datos	PostgreSQL 18, driver <code>pg</code>
Frontend	HTML + JS/TS nativo, Tailwind CSS (solo en build)
Imágenes	Sistema de archivos local en <code>MEDIA_ROOT</code> , fuera de la raíz web
Volumen objetivo	≤ 1 000 000 registros por colección principal

1. CONTRATO DE ENTRADA Y SALIDA

1.1 Tipos base

```
type Id      = String // UUIDv7, generado por PostgreSQL con uuidv7()
type DateISO = String // YYYY-MM-DD
type TimeHHMM = String // HH:MM, 24 h
type Timestamp = String // ISO 8601 con zona horaria (timestampz)
type Email    = String // ≤ 254 chars, minúsculas normalizadas
type Phone    = String // E.164, ≤ 16 chars

enum Role      { ADMIN, DIRECTOR, REGISTRAR, TEACHER }
enum Weekday   { MON, TUE, WED, THU, FRI, SAT }
enum SectionStatus { OPEN, CLOSED, DISSOLVED }
enum EntryType  { FIRST_ENTRY, CONTINUING }
enum EnrollmentStatus { SUBMITTED, APPROVED, WAITLISTED, REJECTED,
WITHDRAWN }
enum ImageKind  { SCHOOL_LOGO, STUDENT_PHOTO,
ENROLLMENT_DOCUMENT }
enum ImageMime  { JPEG, PNG, WEBP }

struct PageRequest { limit: UInt16 /*1..100*/, cursor: Id? }
struct Page<T>     { items: List<T>, nextCursor: Id? }
```

1.2 Autenticación

```
struct User {
  id:      Id
  email:   Email // único
  passwordHash: String // script de node:crypto; nunca sale del servidor
  role:    Role
  teacherId: Id? // obligatorio si role = TEACHER
  active:  Bool
}

struct LoginRequest { email: Email, password: String /*12..128 chars*/ }
struct SessionInfo { userId: Id, role: Role, csrfToken: String, expiresAt: Timestamp }
```

1.3 Estructura académica

```
struct SchoolYear {
  id:      Id
  label:   String // 1..20 chars, único
  startsOn: DateISO
  endsOn:  DateISO // > startsOn
  enrollmentOpensOn: DateISO
  enrollmentClosesOn: DateISO // > enrollmentOpensOn, ≤ endsOn
}
```

```

struct SchoolPolicy {
  yearId:      Id
  maxPerSection: UInt8 // 1..25, default 25
  autoCreateSection: Bool // default true
  smallSectionHint: UInt8? // informativo para el director
  documentRetentionDays: UInt16 // 30..730, default 90
}

struct Grade {
  id:      Id
  yearId: Id
  name:   String // 1..40 chars, único por año
  order:  UInt8  // 0..20
  version: UInt64 // solo lectura; control optimista
}

struct Section {
  id:      Id
  gradeId: Id
  label:   String // 1..10 chars, único por grado
  capacity: UInt8 // 1..maxPerSection, nunca < enrolledCount
  enrolledCount: UInt8 // solo lectura
  status:   SectionStatus
  roomName: String? // ≤ 40 chars
  autoCreated: Bool // solo lectura
  needsTimetable: Bool // solo lectura
}

struct Subject { id: Id, name: String /*1..60 chars, único*/ }

struct Lesson {
  id:      Id
  sectionId: Id
  subjectId: Id
  teacherId: Id
  weekday: Weekday
  startsAt: TimeHHMM
  endsAt:   TimeHHMM // > startsAt
}

```

1.4 Personas

```

struct Teacher { id: Id, fullName: String /*1..120*/, email: Email /*único*/ }

struct Guardian { id: Id, fullName: String /*1..120*/, email: Email /*único*/, phone: Phone }

struct Student {

```

```

id:      Id
fullName: String    // 1..120 chars
birthDate: DateISO
nationalId: String  // 1..30 chars, único
guardianIds: List<Id> // 1..3
}

```

1.5 Catálogos de procedencia

```

struct Locality {
  id:      Id
  name:    String    // 1..80 chars
  district: String?  // ≤ 80 chars; (name, district) único
}

```

```

struct PreviousSchool {
  id:      Id
  name:    String    // 1..120 chars; (name, localityId) único
  localityId: Id
}

```

1.6 Matrícula

```

struct EnrollmentRequest {      // entrada pública
  idempotencyKey: String // 16..64 chars
  yearId:      Id
  gradId:      Id
  student:     { fullName, birthDate, nationalId }
  guardian:    { fullName, email, phone }
  localityId:  Id
  previousSchoolId: Id?
  previousSchoolOther: String? // ≤ 120 chars; excluyente con previousSchoolId
  consentAccepted: Bool // debe ser true
}

```

```

struct EnrollmentReceipt {      // salida pública de submit
  enrollmentId: Id
  status:      EnrollmentStatus
  uploadToken: String // 256 bits; un solo uso por archivo; vence en 60 min
}

```

```

struct CohortKey {              // derivada, no editable
  gradId:      Id
  entryType:   EntryType
  localityId:  Id
  previousSchoolId: Id? // solo FIRST_ENTRY
  previousSectionId: Id? // solo CONTINUING con promoción regular
}

```

```
}
```

```
struct Enrollment {  
  id: Id  
  yearId: Id  
  gradeId: Id  
  studentId: Id  
  entryType: EntryType // resuelto por el servidor  
  previousEnrollmentId: Id?  
  cohortKey: CohortKey? // null hasta normalizar la procedencia  
  sectionId: Id? // null hasta APPROVED  
  status: EnrollmentStatus  
  submittedAt: Timestamp  
  decidedAt: Timestamp?  
  decidedBy: Id?  
  rejectReason: String?  
}
```

```
struct EnrollmentDecision {  
  enrollmentId: Id  
  action: APPROVE | WAITLIST | REJECT  
  sectionId: Id? // null en APPROVE = asignación automática  
  reason: String? // obligatorio en REJECT, ≤ 280 chars  
}
```

1.7 Balanceo de secciones

```
struct SectionOverview {  
  sectionId: Id  
  label: String  
  count: UInt8  
  capacity: UInt8  
  status: SectionStatus  
  needsTimetable: Bool  
  belowHint: Bool  
  cohorts: List<{ cohortKey: CohortKey, count: UInt8 }>  
}
```

```
struct RebalanceRequest {  
  sourceSectionId: Id  
  targetSectionIds: List<Id>? // null = todas las OPEN del grado excepto la origen  
}
```

```
struct StudentMove { enrollmentId: Id, fromSectionId: Id, toSectionId: Id, cohortKey:  
CohortKey }
```

```
struct RebalancePlan {
```

```

planId:      Id
sourceSectionId: Id
moves:      List<StudentMove>
resultingCounts: Map<Id, UInt8>
splitCohorts: List<CohortKey>
lessonsToCancel: List<Id>
basedOnVersion: UInt64
expiresAt:   Timestamp    // 15 min
}

```

1.8 Imágenes

```

struct ImageMeta {
  id:      Id
  kind:    ImageKind
  ownerId: Id          // school_year (logo) | student | enrollment
  mime:    ImageMime    // detectado por magic bytes, no por extensión ni header
  sizeBytes: UInt32      // 1..5 242 880 (5 MiB)
  sha256:  String       // 64 hex
  createdAt: Timestamp
  uploadedBy: Id?       // null si fue una subida pública con uploadToken
}

```

Límites por tipo de imagen

Tipo	Límite
SCHOOL_LOGO	1 por año lectivo; una subida nueva reemplaza la anterior
STUDENT_PHOTO	1 por alumno; una subida nueva reemplaza la anterior
ENROLLMENT_DOCUMENT	hasta 5 por matrícula

1.9 Interfaces de servicio

```

interface AuthService {
  login(LoginRequest)      -> Result<SessionInfo, SchoolError>
  logout()                 -> Result<Unit, SchoolError>
}

```

```

interface AcademicService {
  createYear(SchoolYear sin id)      -> Result<SchoolYear, SchoolError>
  setPolicy(SchoolPolicy)            -> Result<SchoolPolicy, SchoolError>
  createGrade(yearId, name, order)    -> Result<Grade, SchoolError>
  addSection(gradId, label, capacity, roomName?) -> Result<Section, SchoolError>
}

```

```

resizeSection(sectionId, capacity)        -> Result<Section, SchoolError>
closeSection(sectionId)                   -> Result<Section, SchoolError>
createSubject(name)                       -> Result<Subject, SchoolError>
scheduleLesson(Lesson sin id)             -> Result<Lesson, SchoolError>
removeLesson(lessonId)                   -> Result<Unit, SchoolError>
listSections(gradeId, PageRequest)        -> Result<Page<Section>, SchoolError>
timetable(sectionId)                     -> Result<List<Lesson>, SchoolError>
teacherTimetable(teacherId, yearId)       -> Result<List<Lesson>, SchoolError>
}

interface CatalogService {
    addLocality(name, district?)          -> Result<Locality, SchoolError>
    addPreviousSchool(name, localityId)    -> Result<PreviousSchool, SchoolError>
    searchLocalities(prefix, PageRequest)  -> Result<Page<Locality>, SchoolError>
    searchSchools(localityId, prefix, PageRequest) -> Result<Page<PreviousSchool>,
    SchoolError>
}

interface EnrollmentService {
    submit(EnrollmentRequest)             -> Result<EnrollmentReceipt, SchoolError>
    status(enrollmentId, guardianEmail)    -> Result<EnrollmentStatus, SchoolError>
    normalizeOrigin(enrollmentId, previousSchoolId) -> Result<Enrollment, SchoolError>
    decide(EnrollmentDecision)            -> Result<Enrollment, SchoolError>
    withdraw(enrollmentId)                -> Result<Enrollment, SchoolError>
    listByGrade(gradeId, status?, PageRequest) -> Result<Page<Enrollment>, SchoolError>
}

interface SectionBalancingService {
    gradeOverview(gradeId)                -> Result<List<SectionOverview>, SchoolError>
    previewRebalance(RebalanceRequest)     -> Result<RebalancePlan, SchoolError>
    applyRebalance(planId)                -> Result<RebalancePlan, SchoolError>
}

interface ImageService {
    upload(kind, ownerId, stream, declaredSize, uploadToken?) -> Result<ImageMeta,
    SchoolError>
    read(imageId)                        -> Result<(ImageMeta, Stream), SchoolError>
    list(kind, ownerId)                  -> Result<List<ImageMeta>, SchoolError>
    delete(imageId)                     -> Result<Unit, SchoolError>
}

```

1.10 Eventos de dominio

Se registran en la tabla outbox `domain_events` y los despacha el worker.

```

EnrollmentDecided { enrollmentId, status }
SectionAutoCreated { sectionId, gradeId }

```

SectionRebalanced { planId, moves }

1.11 Invariantes

A. Capacidad y secciones

1. **Tope por sección.** Ninguna sección supera `maxPerSection` (≤ 25).
2. **Capacidad flexible por grado.** No hay límite global de alumnos: la capacidad de un grado es la suma de sus secciones `OPEN`.
3. **Aprobación sin sobrecupo.** `APPROVE` incrementa `enrolledCount` con una condición `enrolledCount < capacity` en la misma transacción. Si no hay cupo, devuelve `SECTION_FULL`.
4. **Secciones automáticas.**
 - Reciben el siguiente label alfabético libre (A...Z, AA...), con `capacity = maxPerSection`, `autoCreated = true` y `needsTimetable = true`.
 - Se crean dentro de la transacción de aprobación, con el grado bloqueado (`FOR UPDATE`).
5. **Horario de la sección.** `needsTimetable` se actualiza en `scheduleLesson` y `removeLesson`.

B. Matrícula

1. **Ventana.** `submit` solo se acepta entre `enrollmentOpensOn` y `enrollmentClosesOn`.
2. **Una matrícula activa.** Un alumno tiene como máximo una matrícula no `REJECTED`/`WITHDRAWN` por año.
3. **Idempotencia.** El mismo `idempotencyKey` devuelve el recibo original.
4. **Tipo de ingreso resuelto por el servidor.** Si existe una matrícula `APPROVED` del año anterior con el mismo `nationalId`, el alumno es `CONTINUING`; si no, es `FIRST_ENTRY`.
5. **Procedencia normalizada antes de aprobar.** Si hay `previousSchoolOther`, no se puede aprobar hasta ejecutar `normalizeOrigin`; de lo contrario se devuelve `ORIGIN_NOT_NORMALIZED`.
6. **Transiciones de estado válidas.** Cualquier otra transición devuelve `INVALID_TRANSITION`.

Desde

Hacia

SUBMITTED APPROVED, WAITLISTED, REJECTED,
 WITHDRAWN

WAITLISTED APPROVED, REJECTED, WITHDRAWN

APPROVED **WITHDRAWN** (libera cupo y decrementa contadores)

7. **Cierre de lista de espera.** Con `autoCreateSection = true`, **WAITLISTED** solo ocurre por decisión manual.

C. Agrupación por procedencia

1. **Clave de agrupación por tipo de ingreso.**
 - **FIRST_ENTRY** → (grado, localidad, escuela de procedencia).
 - **CONTINUING** con promoción regular (`order + 1`) → (grado, localidad, sección del año anterior).
 - **CONTINUING** sin promoción regular → (grado, localidad).
2. **Asignación automática.** Se elige la primera opción que cumpla; el desempate siempre es por `label` ascendente.
 - (a) una sección **OPEN** con el mismo **CohortKey** y cupo, eligiendo la de más alumnos de ese cohorte;
 - (b) una sección **OPEN** con la misma localidad y cupo;
 - (c) la sección **OPEN** con menos alumnos y cupo;
 - (d) una sección nueva.
3. **Cohortes grandes.** Un cohorte de más de 25 alumnos llena una sección antes de abrir la siguiente.
4. **Asignación manual.** Si el registrador indica `sectionId`, se respeta aunque rompa la agrupación; solo se valida el cupo.

D. Redistribución

1. **Momento y rol.** Solo después de `enrollmentClosesOn` y solo por **DIRECTOR**. El director elige la sección origen; `smallSectionHint` solo marca candidatas.
2. **Destinos.** Secciones del mismo grado, en estado **OPEN** y distintas de la origen.
3. **Algoritmo.**
 - Los cohortes se reparten enteros, del más grande al más pequeño, hacia la sección destino con menos alumnos que tenga cupo para el cohorte completo.
 - Un cohorte que no cabe entero en ninguna se divide y queda registrado en `splitCohorts`.
 - Objetivo: diferencia máxima de 1 alumno entre destinos, salvo que conservar cohortes enteros lo impida.
4. **Capacidad insuficiente.** Si los alumnos a mover superan el cupo libre total, se devuelve `REBALANCE_INSUFFICIENT_CAPACITY`.
5. **Preview sin efectos.** `previewRebalance` no modifica nada.
6. **Aplicación atómica.** `applyRebalance` mueve las matrículas, actualiza los contadores, marca la origen como **DISSOLVED**, cancela sus lecciones e incrementa `grades.version`, todo en una sola transacción.
 - Si la versión del grado cambió desde el preview, devuelve **PLAN_STALE**.

- Si el plan venció, devuelve `PLAN_EXPIRED`.

E. Imágenes

1. **Almacenamiento.** Los archivos viven en `MEDIA_ROOT/{kind}/{yyyy}/{mm}/{id}.{ext}`. El nombre original del archivo nunca se usa ni se almacena.
2. **Validación del tipo.** El tipo se determina solo por magic bytes; cualquier otro formato devuelve `IMAGE_TYPE_NOT_ALLOWED`.

Formato Magic bytes

JPEG `FF D8 FF`

PNG `89 50 4E
47`

WebP `RIFF....W
EBP`

3. **Tamaño.** Se lee el stream con un contador; al superar 5 MiB se aborta y se devuelve `IMAGE_TOO_LARGE`, sin esperar a recibir el archivo completo.
4. **Metadatos.** El servidor elimina los metadatos EXIF/XMP mediante parsing nativo de segmentos: APP1 en JPEG, chunks `eXIf/tEXt/iTXt` en PNG, chunks `EXIF/XMP` en WebP. Si la estructura no es parseable, devuelve `IMAGE_INVALID`. Así ninguna foto de un menor conserva GPS ni datos del dispositivo.
5. **Escritura atómica.**
 - Se escribe en `MEDIA_ROOT/.tmp/{id}` mientras se calcula el sha256 en streaming.
 - Luego `fsync`, `rename` al destino final e `INSERT` de metadatos.
 - Si el `INSERT` falla, se elimina el archivo.
6. **Subida pública.** Solo para `ENROLLMENT_DOCUMENT`, con el `uploadToken` del recibo de matrícula. El token vence en 60 minutos y permite hasta 5 subidas.
7. **Lectura.** Siempre por la ruta HTTP con control de rol; `MEDIA_ROOT` nunca se sirve como estático. El logo es la única imagen pública.
8. **Borrado.** Es lógico (`deletedAt`), y el worker elimina el archivo físico. Los documentos de matrículas `REJECTED/WITHDRAWN` se purgan tras `documentRetentionDays`.

2. CONTRATO HTTP

2.1 Rutas

Todas las respuestas son JSON, salvo la lectura de imágenes. Las subidas usan cuerpo binario crudo (**Content-Type: image/jpeg|png|webp**), no multipart.

Método	Ruta	Acceso	Operación
POST	/api/auth/login	público	login
POST	/api/auth/logout	sesión	logout
POST	/api/public/enrollments	público	submit
POST	/api/public/enrollments/status	público	status
PUT	/api/public/enrollments/{id}/documents	X-Upload-Token	upload(ENROLLMENT_DOCUMENT)
GET	/api/public/catalog/localities	público	searchLocalities
GET	/api/public/catalog/localities/{id}/schools	público	searchSchools
GET	/api/public/images/logo?yearId=	público	logo
POST	/api/catalog/localities · /api/catalog/schools	REGISTRAR, ADMIN	catálogo
GET	/api/grades/{id}/enrollments	REGISTRAR, DIRECTOR	listByGrade
POST	/api/enrollments/{id}/normalize-origin	REGISTRAR	normalizeOrigin
POST	/api/enrollments/{id}/decision	REGISTRAR	decide
POST	/api/enrollments/{id}/withdraw	REGISTRAR	withdraw
POST	/api/years · PUT /api/years/{id}/policy	ADMIN	año y política

POST	/api/grades /api/grades/{id}/sections	ADMIN	createGrade addSection
PATCH	/api/sections/{id}	ADMIN	resizeSection closeSection
POST	/api/subjects	ADMIN	createSubject
POST	/api/sections/{id}/lessons	ADMIN	scheduleLesson
DELETE	/api/lessons/{id}	ADMIN	removeLesson
GET	/api/grades/{id}/sections	ADMIN, DIRECTOR	listSections
GET	/api/sections/{id}/timetable	ADMIN, DIRECTOR , TEACHER	timetable
GET	/api/teachers/{id}/timetable? yearId=	ADMIN, TEACHER (solo el propio)	teacherTimetable
GET	/api/grades/{id}/overview	DIRECTOR	gradeOverview
POST	/api/rebalance/preview	DIRECTOR	previewRebalance
POST	/api/rebalance/{planId}/apply	DIRECTOR	applyRebalance
PUT	/api/images/{kind}/{ownerId}	ADMIN, REGISTRAR	upload
GET	/api/images/{id}	ADMIN, REGISTRAR, DIRECTOR	read
GET	/api/images?kind=&ownerId=	ADMIN, REGISTRAR, DIRECTOR	list
DELETE	/api/images/{id}	ADMIN, REGISTRAR	delete

2.2 Reglas HTTP

1. Sesión.

- Cookie `sid` con `HttpOnly; Secure; SameSite=Strict`, que expira a las 8 h sin actividad.
- Toda mutación autenticada exige además el header `X-CSRF-Token`.

2. Rate limit por IP.

Ruta	Límite
<code>login</code>	5/min
<code>submit</code> y <code>status</code>	10/min
Subidas públicas	5/min

3. **Enumeración.** `status` devuelve un `404` idéntico si la matrícula no existe o si el email no coincide. `login` devuelve siempre `INVALID_CREDENTIALS` sin distinguir la causa.
 4. **Lectura de imágenes.** La respuesta incluye `Content-Type` real, `X-Content-Type-Options: nosniff`, `Cache-Control: private, max-age=300` y `Content-Disposition: inline`.
 5. **Headers de seguridad globales.** `Content-Security-Policy: default-src 'self'`, `Referrer-Policy: same-origin` y `X-Frame-Options: DENY`.
 6. **Paginación.** Solo por cursor.
 7. **Formato de error.** `{ error: SchoolError, fields?: List<String> }`.
-

3. CONTRATO DE PERSISTENCIA (PostgreSQL 18 + sistema de archivos)

3.1 Tablas

Todas las claves primarias `id` usan `uuid DEFAULT uuidv7()` y todas las fechas usan `timestamptz`.

Tabla	Únicos	Índices de consulta	CHECK / notas
<code>users</code>	<code>email</code>	—	<code>role = 'TEACHER' ⇒</code>

			teacher_id no nulo
sessions	id (256 bits aleatorio)	user_id, expires_at	purga por TTL
rate_limits (UNLOGGED)	(ip, route, window_start)	window_start	purga cada 60 s
school_years	label	—	fechas ordenadas
school_policies	PK year_id	—	max_per_section BETWEEN 1 AND 25
grades	(year_id, name)	(year_id, "order")	version bigint
sections	(grade_id, label)	(grade_id, status)	enrolled_count <= capacity, capacity BETWEEN 1 AND 25
subjects	name	—	—
lessons	—	(section_id, weekday, starts_at), (teacher_id, weekday, starts_at)	ends_at > starts_at; el traslape se valida en la transacción
teachers	email	—	—
guardians	email	—	—
students	national_id	—	—
student_guardians	PK (student_id, guardian_id)	guardian_id	máximo 3 por alumno, validado en la aplicación
localities	(name, district)	name text_pattern_ops (prefijo)	—

previous_schools	(name, locality_id)	(locality_id, name, text_pattern_ops)	—
enrollments	idempotency_key; (year_id, student_id) WHERE status NOT IN ('REJECTED', 'WITHDRAWN')	(grade_id, status, submitted_at), (year_id, student_id, status), section_id	CohortKey en columnas planas: entry_type, locality_id, previous_school_id, previous_section_id
section_cohort_counts	PK (section_id, cohort_hash)	(grade_id, cohort_hash)	cohort_hash = sha256 del CohortKey canónico
rebalance_plans	PK id	expires_at	moves en jsonb; purga por TTL
images	(kind, owner_id) WHERE kind <> 'ENROLLMENT_DOCUMENT' AND deleted_at IS NULL	(kind, owner_id), deleted_at	size_bytes BETWEEN 1 AND 5242880
upload_tokens	PK token_hash	expires_at	se guarda solo el sha256 del token; uses_left 0..5
domain_events	PK id	dispatched_at IS NULL (índice parcial)	outbox

3.2 Proyecciones obligatorias en rutas calientes

Operación	Columnas
listSections	id, label, capacity, enrolled_count, status, needs_timetable
timetable / teacherTimetable	id, subject_id, teacher_id, weekday, starts_at, ends_at

<code>listByGrade</code>	<code>id, student_id, status, submitted_at, section_id</code>
<code>status</code> público	<code>status</code> , más la verificación del email del tutor por join indexado
Resolución de continuidad	<code>id, section_id, grade_id</code> por (<code>year_id, student_id, status</code>)
<code>gradeOverview</code>	<code>sections</code> + <code>section_cohort_counts</code> , sin datos personales
<code>previewRebalance</code>	<code>id, section_id</code> y columnas de cohorte de las matrículas <code>APPROVED</code> de la sección origen

3.3 Reglas de persistencia

1. **Contadores.** `enrolled_count`, `section_cohort_counts`, `needs_timetable` y `grades.version` se actualizan en la misma transacción que los provoca. No se usan triggers ni recálculos con `COUNT(*)`.
2. **Sin borrado físico** de años, grados, secciones, matrículas ni personas. Se usan los estados `CLOSED` y `DISSOLVED` y las transiciones de matrícula.
3. **Worker** (`src/jobs.ts`, proceso aparte). Sus tareas:
 - Purgar las filas vencidas de `sessions`, `rate_limits`, `rebalance_plans` y `upload_tokens`.
 - Despachar `domain_events`.
 - Borrar físicamente las imágenes con `deleted_at`.
 - Aplicar la retención de documentos.
 - Barrer huérfanos: archivos sin fila con más de 24 h se eliminan; filas sin archivo generan telemetría.
4. **Respaldo.** El respaldo es un conjunto indivisible: `pg_dump` junto con la copia de `MEDIA_ROOT`, tomada en la misma ventana.
5. **Permisos de `MEDIA_ROOT`.** Debe estar fuera del directorio servido, con permisos `0700` para el usuario del proceso Node.

4. CONTRATO DE ERRORES

```
enum SchoolError {
  // Validación y existencia
  VALIDATION_FAILED, CONSENT_REQUIRED, NOT_FOUND, DUPLICATE,
  UNKNOWN_LOCALITY, UNKNOWN_PREVIOUS_SCHOOL,
  SCHOOL_LOCALITY_MISMATCH,
  // Autenticación y acceso
```

```

    UNAUTHENTICATED, INVALID_CREDENTIALS, FORBIDDEN, CSRF_INVALID,
    RATE_LIMITED,
    // Matrícula
    ENROLLMENT_WINDOW_CLOSED, ALREADY_ENROLLED,
    ORIGIN_NOT_NORMALIZED, INVALID_TRANSITION,
    // Secciones y horario
    SECTION_FULL, SECTION_CLOSED, CAPACITY_BELOW_ENROLLED,
    CAPACITY_ABOVE_POLICY, SCHEDULE_CONFLICT,
    // Redistribución
    ENROLLMENT_STILL_OPEN, INVALID_REBALANCE_TARGET,
    REBALANCE_INSUFFICIENT_CAPACITY,
    PLAN_STALE, PLAN_EXPIRED,
    // Imágenes
    IMAGE_TOO_LARGE, IMAGE_TYPE_NOT_ALLOWED, IMAGE_INVALID,
    IMAGE_LIMIT_REACHED, UPLOAD_TOKEN_INVALID,
    // Infraestructura
    STORAGE_FAILURE           // base de datos o sistema de archivos; causa nativa solo en
    telemetría
}

```

HTTP

Errores

```

400    VALIDATION_FAILED, CONSENT_REQUIRED, IMAGE_INVALID

401    UNAUTHENTICATED, INVALID_CREDENTIALS, UPLOAD_TOKEN_INVALID

403    FORBIDDEN, CSRF_INVALID

404    NOT_FOUND, UNKNOWN_LOCALITY, UNKNOWN_PREVIOUS_SCHOOL

409    DUPLICATE, ALREADY_ENROLLED, ORIGIN_NOT_NORMALIZED,
        INVALID_TRANSITION, SECTION_FULL, SECTION_CLOSED,
        CAPACITY_BELOW_ENROLLED, CAPACITY_ABOVE_POLICY,
        SCHEDULE_CONFLICT, SCHOOL_LOCALITY_MISMATCH,
        INVALID_REBALANCE_TARGET, REBALANCE_INSUFFICIENT_CAPACITY,
        PLAN_STALE, IMAGE_LIMIT_REACHED

410    PLAN_EXPIRED

413    IMAGE_TOO_LARGE

415    IMAGE_TYPE_NOT_ALLOWED

422    ENROLLMENT_WINDOW_CLOSED, ENROLLMENT_STILL_OPEN

429    RATE_LIMITED

```

5. ANTI-BLOAT FENCE

Fuera de alcance

- Cobros y pagos.
- Calificaciones y asistencia.
- Expediente médico.
- Mensajería (un consumidor externo lee los eventos).
- Multi-escuela.
- Cuentas de tutores.

Campos descartados

- Dirección exacta, religión, datos médicos y datos socioeconómicos.
- Coordenadas geográficas.
- Nombre original de los archivos subidos.
- Dimensiones de las imágenes.

Abstracciones descartadas

- ORMs y query builders.
- Motor de reglas y motor de workflows (la máquina de estados es un `enum` más una tabla de transiciones).
- Campos dinámicos (EAV).
- Solvers de optimización.
- Geocodificación y fuzzy-matching.
- Redistribución automática sin intervención del director.
- Historial de continuidad más allá del año inmediato anterior.

Dependencias de runtime permitidas: `pg`. Nada más.

Dependencias de desarrollo permitidas: `tailwindcss`, `typescript` (solo `tsc --noEmit`) y `@types/node`.

Paquetes prohibidos

Categoría	Prohibidos
Frameworks HTTP	Express, Fastify, Koa
Acceso a datos	Prisma, TypeORM, Sequelize, Knex
Validación	Zod, Joi

Seguridad	bcrypt (se usa scrypt de <code>node:crypto</code>), helmet, csurf, express-session
Subidas	multer, busboy, formidable
Imágenes	sharp, jimp, exif-parser
Fechas	moment, dayjs
Identificadores	uuid
Infraestructura	Redis, brokers de mensajes, librerías de rate limiting
Frontend	librerías de estado global, kits de UI, captcha de terceros (se usa un honeypot)

Telemetría por excepción

Solo se registran:

- `STORAGE_FAILURE`;
- `RATE_LIMITED` sostenido por IP;
- `CSRF_INVALID`;
- `INVALID_CREDENTIALS` repetido por IP;
- `PLAN_STALE` repetido;
- archivos o filas de imágenes huérfanos.

Los logs nunca incluyen `nationalId`, `birthDate`, contacto, localidad junto a un identificador de alumno, tokens ni hashes de contraseña.

Pendientes antes de `/znve -execute`

1. **Tipos de imagen.** ¿Se confirman `SCHOOL_LOGO`, `STUDENT_PHOTO` y `ENROLLMENT_DOCUMENT`? ¿Hay otros? La foto del alumno reincorpora un dato que v1.1 había descartado.
2. **Formato de documentos.** Los documentos de matrícula solo aceptan imagen. ¿Se necesita también PDF?
3. **Ubicación de `MEDIA_ROOT`.** Falta la ruta en el servidor; por ejemplo, `/var/lib/escuela/media`.